

Wydział Informatyki

Katedra Systemów Inteligentnych i Data Science

Inteligentne systemy przetwarzania danych

Kamil Czyżewski s22370

Jakub Szulc s23640

Konfigurowalne środowisko AutoML dla
finansowych potoków predykcyjnych w
turnieju Numerai

Praca magisterska

Promotor techniczny Filip Dziecioł

Promotor główny Grzegorz Wójcik

Warszawa, 2026

Faculty of Computer Science

Department of Intelligent Systems and Data Science

Kamil Czyżewski s22370

Jakub Szulc s23640

A Configurable AutoML Framework for Financial Prediction Pipelines in the Numerai Tournament

Master's thesis

Technical supervisor Filip Dziecioł

Lead supervisor Grzegorz Wójcik

Warsaw, 2026

Abstrakt

AlphaPulse to konfigurowalne środowisko AutoML, które zbudowaliśmy pod turniej Numerai: od deklaratywnego YAML-a, przez routing cech i preprocessing, modele tabelaryczne i HPO (Optuna albo Ray Tune), po walidację z erami, rejestr artefaktów, diagnostykę Weights & Biases i eksport przenośnej funkcji predykcyjnej.

Sprawdziliśmy system na dwóch torach. Najpierw pełnoskalowy przebieg na 2,75 mln wierszy uczących. Potem końcową serię HPO na danych Numerai v5.3 dla celu `target_ender_60`: 27 kompletnych konfiguracji na stałej próbie 274 627 wierszy, z 16 erami separacji, 52 erami holdoutu i późniejszym oknem 88 er z Meta Modelem. Na holdoucie wygrała próba 11 (LightGBM): CORR Sharpe 1,044, średni CORR 0,0374. Na późniejszym oknie próba 22 miała najwyższy CORR Sharpe (0,425), a próba 4 najwyższy MMC Sharpe (0,306). To nie jeden zwycięzca, tylko mały front Pareto. W praktyce AlphaPulse domyka cykl od surowych danych do porównywalnych artefaktów i zostawia sensownych kandydatów pod pełny trening produkcyjny.

Słowa kluczowe: uczenie maszynowe, AutoML, Numerai, walidacja czasowa, optymalizacja hiperparametrów

Abstract

AlphaPulse is a configurable AutoML stack we built for Numerai. You declare an experiment in YAML. The runtime then wires feature routing, preprocessing, tabular models, Optuna or Ray Tune search, era-aware splits, artefact logging, Weights & Biases diagnostics and a portable prediction export.

We checked it twice. First came a full-scale run on 2.75 million training rows. Then a final HPO study on Numerai v5.3 for `target_ender_60`: 27 complete configurations on a fixed sample of 274 627 rows, a 16-era gap, a 52-era holdout and a later 88-era window aligned with Meta Model predictions. On holdout, trial 11 (LightGBM) won with CORR Sharpe 1,044 and mean CORR 0,0374. Later, trial 22 led on CORR Sharpe (0.425) while trial 4 led on MMC Sharpe (0.306). Not one winner, but a small Pareto set. In practice the pipeline takes a run from raw data to comparable artefacts and leaves candidates worth a full production retrain.

Keywords: machine learning, AutoML, Numerai, temporal validation, hyperparameter optimisation

Spis treści

1	Wstęp	5
1.1	Kontekst i motywacja	5
1.2	Problem badawczy	5
1.3	Cele pracy	6
1.4	Zakres i ograniczenia	7
1.5	Wkład autorów	7
1.6	Kryteria sukcesu	8
2	Podstawy problemu i kontekst systemu AlphaPulse	9
2.1	Specyfika turnieju Numerai	9
2.2	Charakterystyka danych finansowych	9
2.3	Metryki jakości i kryteria oceny	10
2.4	Ryzyko wycieku informacji i poprawna walidacja	11
2.5	AutoML, HPO i modele tabelaryczne	12
2.6	Luka projektowa i pozycjonowanie AlphaPulse	12
2.7	Zakres funkcjonalny AlphaPulse jako punkt odniesienia dalszych rozdziałów	14
2.8	Mapowanie wyzwań domenowych na decyzje projektowe	15
2.9	Wnioski	15
3	Architektura systemu AlphaPulse	16
3.1	Założenia architektoniczne	16
3.2	Widok warstwowy systemu	16
3.3	Przepływ wykonania end-to-end	18
3.4	Komponenty funkcjonalne i ich odpowiedzialności	19
3.5	Wzorce zapewniające jakość inżynierską	19
3.6	Ograniczenia architektury	19
3.7	Diagram architektury logicznej	20
3.8	Kontrakty komponentów	20
4	Implementacja systemu AlphaPulse	22
4.1	Organizacja repozytorium	22
4.2	Warstwa uruchomieniowa (CLI)	22
4.3	Konfiguracja eksperymentów (YAML)	23
4.4	Implementacja warstwy danych	24
4.5	Implementacja pipeline’u modelowego	25
4.6	Implementacja HPO	26
4.7	Implementacja AutoResearch	27
4.8	Eksport predykcji i artefakty końcowe	27
4.9	Zapewnienie jakości kodu	28
4.10	Przykładowa konfiguracja referencyjna	28
4.11	Przestrzeń przeszukiwania HPO	29

5	Metodyka badań	30
5.1	Pytania eksperymentalne	30
5.2	Dwie serie o rozłącznych rolach	30
5.3	Dane i podział czasowy badania końcowego	31
5.4	Protokół HPO	32
5.5	Przestrzeń konfiguracji	32
5.6	Metryki i reguła selekcji	33
5.7	Metody analizy	33
5.8	Reprodukowalność	34
5.9	Zagrożenia dla trafności	34
6	Wyniki eksperymentów	35
6.1	Weryfikacja pełnej skali	35
6.2	Kompletność końcowej serii HPO	35
6.3	Rozkład oficjalnych metryk	36
6.4	Rodziny modeli w ustalonym budżecie	36
6.5	Przebieg optymalizacji	37
6.6	Transfer między przedziałami czasowymi	39
6.7	Kandydaci końcowi	40
6.8	Synteza wyników	41
7	Dyskusja i zakończenie	42
7.1	Realizacja celu pracy	42
7.2	Interpretacja wyników modelowych	42
7.3	Odpowiedzi na pytania eksperymentalne	43
7.4	Znaczenie inżynierskie	43
7.5	Ograniczenia	44
7.6	Rekomendacja wdrożeniowa i dalszy rozwój	44
7.7	Wnioski końcowe	44
A	Materiały odtworzeniowe	46
A.1	Identyfikacja protokołu	46
A.2	Receptury kandydatów	46
A.3	Granice odtworzenia wyniku	47
	Bibliografia	48

Rozdział 1

Wstęp

1.1 Kontekst i motywacja

Prognozowanie rynków finansowych jest trudne nawet jak na standardy uczenia maszynowego. Sygnał jest słaby względem szumu, szeregi są niestacjonarne, obserwacje silnie zależą od czasu, a dane układają się w ery (tygodniowe przekroje rynku). Przy takiej strukturze losowy split train/test szybko wprowadza w błąd. Ocena na kolejnych okresach pomaga, ale nie kasuje ryzyka, że przeuczymy się na samym procesie selekcji albo że rynek po prostu się zmieni [25, 24].

Numerai, start z 2015 roku (Numerai Inc.), to dość nietypowa platforma pod finansowe ML. Uczestnicy dostają zaszyfrowane, zanonimizowane tabele ze spółkami i ich atrybutami. Trzeba zbudować model pod jeden z celów (u nas `target_ender_60`) i oddać prognozy alpha. Fundusz zbiera wszystkie submisje w Meta Model, którym realnie handluje. Dodatkowo można stakować NMR: dobre modele zarabiają, słabe tracą [5].

Stąd praktyczne pytanie: jak systematycznie kręcić konfiguracjami ML, żeby poprawiać korelację per-era i Sharpe’a, nie psując formatu submisji i nie wpuszczając wycieku czasowego?

Zwykły AutoML dobiera modele i hiperparametry, ale nie ogarnia sam z siebie er, purgingu zależnego od horyzontu, oficjalnych metryk Numerai ani eksportu `predict.pkl` [10]. AlphaPulse spina te elementy w jeden przepływ: dane, YAML, trening, ocena, artefakt.

1.2 Problem badawczy

Praca odpowiada na następujące pytanie badawcze:

Jak zaprojektować, zaimplementować i zweryfikować środowisko, które umożliwi systematyczne i reprodukowalne eksperymentowanie z potokami uczenia maszynowego dla danych finansowych turnieju Numerai?

Problem ten dekomponuje się na następujące pytania szczegółowe:

1. Jakie kontrakty komponentów są potrzebne, aby połączyć dane, preprocessing, modele, strategie zespołowe, ewaluację i eksport w jednym systemie?

2. Jak zakodować wymagania walidacji czasowej i reprodukowalności tak, aby kolejne konfiguracje były oceniane według tego samego protokołu?
3. Czy zaimplementowany potok wykonuje kompletny przebieg na pełnej skali danych i pozostawia artefakty umożliwiające audyt uruchomienia?
4. Jak w ustalonym budżecie zachowują się badane rodziny modeli oraz w jakim stopniu ranking z holdoutu przenosi się na późniejszy przedział czasowy?

1.3 Cele pracy

Chcemy zbudować AlphaPulse tak, żeby:

- ujednolicił pełny przepływ pracy konkursu Numerai - od pobierania danych po eksport submisji
- umożliwił deklaratywne definiowanie eksperymentów w formacie YAML, bez konieczności modyfikowania kodu źródłowego
- zapewniał poprawną walidację z uwzględnieniem struktury er, purging i embargo
- automatyzował poszukiwanie optymalnych hiperparametrów i konfiguracji pipeline'u za pomocą Optuna (tryb `--local`) lub Ray Tune (tryb rozproszony), z obsługą `--resume` i bazą TrialDB
- udostępniał rozszerzenie AutoResearch, w którym model językowy proponuje modyfikacje konfiguracji, bez przypisywania temu mechanizmowi przewagi empirycznej w badaniu końcowym
- raportował oficjalnie zdefiniowane CORR i MMC oraz ich agregaty per-era
- eksportował wyniki w postaci pliku `predict.pkl` zgodnego z wymaganiami API Numerai oraz wspierał tygodniowy cykl produkcyjny: inferencja na `live.parquet` i automatyczna submisja predykcji

Celami pomocniczymi są: weryfikacja skalowalności kompletnego przepływu, opisowe porównanie rodzin modeli w jednakowym protokole HPO, analiza zgodności rankingów na dwóch kolejnych przedziałach czasowych oraz wskazanie kandydatów reprezentujących kompromis CORR-MMC. Praca nie mierzy oszczędności czasu człowieka, przewagi AutoResearch ani przyczynowego wpływu pojedynczych komponentów.

1.4 Zakres i ograniczenia

Framework AlphaPulse jest przeznaczony dla turnieju Numerai Classic, a badanie końcowe wykorzystuje zestaw danych w wersji v5.3. Zakres pracy obejmuje:

- Modele tabelaryczne: gradient boosting, modele drzewiste, modele liniowe, fundamentalne modele tabelaryczne (TabPFN, TabPFN3, TabICL) oraz uczenie głębokie (TabularDL z architekturami FT-Transformer i MLP)
- Preprocessory: skalowanie, redukcja wymiarowości, selekcja cech, autoencoder, kompresja cech, selektor stabilności per-era (EraStableFeatureSelector) i regularyzacja poprzez szum gaussowski
- Strategie ensemble: **single**, **weighted**, **stacking**; potoki wielogłowicowe (MultiHeadPipeline) i wielocelowe (MultiTargetPipeline)
- Optymalizację hiperparametrów (Optuna TPE w trybie lokalnym, Ray Tune w trybie rozproszonym) oraz system AutoResearch z agentem opartym na modelu językowym
- Panel EDA (Streamlit) do eksploracji danych i wizualizacji wyników HPO

Praca nie obejmuje: analizy modeli sekwencyjnych, czyli na przykład LSTM czy Transformer, dla danych czasowych, strategii tradingowych ani mechanizmów stakowania tokenów NMR, a także integracji z turniejem Numerai Signals.

1.5 Wkład autorów

Praca powstała wspólnie, przy następującym głównym podziale odpowiedzialności:

- Jakub Szulc odpowiadał przede wszystkim za architekturę i implementację rdzenia AlphaPulse, potoki modelowe, warstwę HPO, reprodukowalność uruchomień, eksperymenty końcowe oraz integrację eksportu i inferencji
- Kamil Czyżewski odpowiadał przede wszystkim za panel analizy eksploracyjnej, przygotowanie i interpretację wizualizacji, dokumentację systemu oraz redakcję i skład pracy
- Obaj autorzy uczestniczyli w definiowaniu zakresu systemu, przeglądzie wyników, weryfikacji poprawności metryk i formułowaniu wniosków

Repozytorium i historia wersji dokumentują ewolucję implementacji oraz wkład autorów [1].

1.6 Kryteria sukcesu

Praca uznaje się za spełnioną, jeśli:

1. framework umożliwia powtarzalne uruchomienia tej samej konfiguracji (seed globalny, artefakt `*_provenance.json`)
2. HPO poprawnie realizuje próbkowanie, ocenę, persystencję i wznowienie prób
3. mechanizm AutoResearch poprawnie proponuje mutacje oraz zapisuje stan umożliwiający wznowienie, przy czym jego skuteczność nie jest przedmiotem porównania empirycznego
4. końcowy artefakt submisji przechodzi walidację formatu i test dymny (*export smoke test*)
5. seria eksperymentalna identyfikuje co najmniej jedną konfigurację o dodatnim CORR Sharpe na holdoucie i dodatnim MMC Sharpe na późniejszym przedziale selekcyjnym

Rozdział 2

Podstawy problemu i kontekst systemu AlphaPulse

W tym rozdziale zbieramy kontekst domenowy i techniczny, w którym powstał *AlphaPulse*: turniej Numerai, charakter danych, kryteria oceny modeli, ograniczenia metodologiczne oraz wymagania wobec AutoML dla finansowych potoków predykcyjnych.

2.1 Specyfika turnieju Numerai

Numerai jest konkursem data science, w którym uczestnicy budują modele predykcyjne na zanonimizowanych danych tabelarycznych i dostarczają predykcje do wspólnego meta-modelu funduszu. W przeciwieństwie do klasycznych konkursów ML liczy się jakość predykcji, ale też stabilność sygnału i odporność na zmianę warunków rynkowych [2].

Przekłada się to na kilka konsekwencji:

- dane mają strukturę czasową i grupową (ery)
- błędna walidacja może sztucznie zawyżać wyniki
- reprodukowalność eksperymentów ma realne znaczenie
- końcowym artefaktem jest plik predykcji w wymaganym formacie submisji

Takie środowisko premiuje podejście systemowe: powtarzalne pipeline'y, kontrolę konfiguracji, jasne logowanie eksperymentów i automatyzację przeszukiwania przestrzeni modeli.

2.2 Charakterystyka danych finansowych

Dane używane w zadaniach Numerai to dane tabelaryczne z wyraźnym komponentem czasowym. Z perspektywy modelowania najważniejsze są:

- niski stosunek sygnału do szumu
- niestacjonarność rozkładów cech i celu
- zależności temporalne między obserwacjami

- podatność na przeciek informacji przy niepoprawnym podziale danych

Dlatego i preprocessing, i procedura walidacyjna muszą być projektowane ostrożnie. Szczególnie ważne są metody uwzględniające strukturę er oraz separację czasową między zbiorem uczącym a walidacyjnym. Zmiana rozkładu między okresem uczenia i przyszłą oceną dodatkowo ogranicza możliwość przenoszenia wyników bez kontroli stabilności w czasie [26].

2.3 Metryki jakości i kryteria oceny

W odróżnieniu od wielu zadań regresyjnych, ocena modeli finansowych nie powinna opierać się wyłącznie na błędzie punktowym. Dla każdej ery e oficjalny CORR porównuje predykcje $\hat{y}_e$ z celem y_e po transformacjach określonych w dokumentacji Numerai [3]. Niech n_e oznacza liczbę obserwacji w erze, $R_e(v_i)$ rangę elementu v_i z zachowaniem remisów, a Φ^{-1} kwantyl standardowego rozkładu normalnego. Transformację predykcji można zapisać jako

$$z_e(v_i) = \Phi^{-1}\left(\frac{R_e(v_i) - \frac{1}{2}}{n_e}\right), \quad g(x) = \text{sgn}(x)|x|^{3/2}. \quad (2.1)$$

Oficjalny wynik CORR dla ery jest następnie korelacją Pearsona przekształconych wektorów:

$$c_e = \rho_P(g(y_e - \bar{y}_e), g(z_e(\hat{y}_e))). \quad (2.2)$$

MMC mierzy część sygnału predykcji, której nie wyjaśnia Meta Model. Dla $p_e = z_e(\hat{y}_e)$ oraz znormalizowanej predykcji Meta Modelu $b_e = z_e(b_e^{\text{raw}})$ neutralizacja jest rzutem na dopełnienie ortogonalne wektora b_e :

$$p_e^\perp = p_e - b_e \frac{p_e^\top b_e}{b_e^\top b_e}. \quad (2.3)$$

Dla celu Numerai zapisanego w przedziale $[0, 1]$ wynik wkładu w erze ma postać

$$m_e = \frac{1}{n_e} (4y_e - \overline{4y_e})^\top p_e^\perp. \quad (2.4)$$

Definicja odpowiada funkcji `correlation_contribution` z zamrożonej wersji `numeraï-tools` 0.6.0 [4, 6].

Stabilność obu miar agreguje współczynnik Sharpe’a liczony po erach:

$$S_{\text{CORR}} = \frac{\bar{c}}{s_c}, \quad S_{\text{MMC}} = \frac{\bar{m}}{s_m}, \quad (2.5)$$

gdzie $c = (c_1, \dots, c_E)$ i $m = (m_1, \dots, m_E)$, a s_c i s_m są odchyleniami standardowymi między erami. AlphaPulse raportuje także:

- średni CORR i odchylenie standardowe między erami
- udział er z dodatnim CORR oraz maksymalne obsunięcie skumulowanego CORR
- średni MMC, jego odchylenie standardowe i S_{MMC}

Implementacje zweryfikowano względem tej samej zamrożonej wersji biblioteki. W badaniu głównym kryterium optymalizacji stanowi S_{CORR} na holdoucie oddzielonym strefą purge. CORR i MMC na późniejszym przedziale selekcyjnym służą do analizy transferu w czasie oraz wyboru kompromisu między siłą i unikalnością sygnału. Ponieważ ten drugi przedział uczestniczy w wyborze kandydatów, raportowane na nim wartości nie są bezstronnym oszacowaniem przyszłego wyniku produkcyjnego [24, 23].

2.4 Ryzyko wycieku informacji i poprawna walidacja

W modelach finansowych jednym z najpoważniejszych problemów jest *look-ahead bias*: niejawne użycie informacji, której w momencie predykcji jeszcze by nie było. Może pojawić się na wielu etapach:

- podczas przygotowania cech
- przy podziale danych na train/validation
- przy doborze hiperparametrów

Dlatego system klasy *AlphaPulse* musi wspierać walidację zgodną z naturą danych czasowych i erowych, w tym podejścia *era-aware* oraz *purged cross-validation*. Wymóg ten stanowi fundament wiarygodnej oceny i bezpośrednio wpływa na konstrukcję architektury frameworku [7, 22]. Sama separacja czasowa nie usuwa jednak błędu selekcyjnego. Wielokrotna optymalizacja na tym samym holdoucie może dopasować konfigurację do szumu kryterium, dlatego wynik wybranej próby należy interpretować warunkowo względem całej procedury wyszukiwania [24, 25].

2.5 AutoML, HPO i modele tabelaryczne

Systemy AutoML automatyzują dobór algorytmu, transformacji i hiperparametrów, ale ich skuteczność zależy od poprawnej definicji przestrzeni poszukiwań, budżetu i protokołu walidacji [10, 18, 19]. Losowe wyszukiwanie stanowi naturalny punkt odniesienia dla metod adaptacyjnych, ponieważ efektywny wymiar przestrzeni hiperparametrów bywa znacznie mniejszy od jej wymiaru nominalnego [20]. AlphaPulse wykorzystuje Optuna TPE do sekwencyjnego wyboru prób oraz Ray Tune do wykonania rozproszonego [8, 9]. Warstwa optymalizacji nie zastępuje metodyki badawczej: każda konfiguracja przechodzi ten sam podział czasowy, ten sam scoring i zapis proveniencji.

Przestrzeń modeli obejmuje algorytmy gradient boosting - XGBoost, LightGBM i CatBoost - które pozostają silnymi punktami odniesienia dla danych tabelarycznych [11, 12, 13, 17]. Uzupełniają je modele liniowe, zespoły drzew oraz fundamentalne modele tabelaryczne, w tym TabPFN i TabICL [15, 16]. Z perspektywy badawczej ważna jest wspólna abstrakcja modelu: umożliwia porównanie rodzin w jednym protokole, bez wiązania warstwy danych i ewaluacji z konkretną biblioteką.

2.6 Luka projektowa i pozycjonowanie

AlphaPulse

Ogólny framework AutoML zazwyczaj kończy działanie na wytrenowanym estymatorze. Tracker eksperymentów rejestruje przebieg, lecz nie definiuje podziału czasowego ani formatu wdrożenia. AlphaPulse łączy te role z wymaganiami Numerai, co przedstawia tabela 2.1.

Podejście *configuration-driven* rozdziela opis eksperymentu od mechaniki wykonania. Dzięki temu konfiguracja jest wersjonowalna, a ten sam graf potoku można uruchomić w trybie pojedynczym, HPO, diagnostycznym i eksportowym. Utrwalenie kodu, danych, zależności i parametrów odpowiada praktykom reprodukowalnego raportowania badań ML [21].

Tabela 2.1: Pozycjonowanie AlphaPulse względem klas narzędzi wspierających eksperymenty ML

Klasa rozwiązania	Automatyzacja konfiguracji	Walidacja erowa	Eksport Numerai
Ogólny AutoML	wysoka	wymaga konfiguracji domenowej	brak
Tracker eksperymentów	nie	nie	brak
Skrypt konkursowy	zależna od implementacji	możliwa	zwykle pojedynczy format
AlphaPulse	HPO i AutoResearch	purge, holdout i walk-forward	<code>predict.pkl</code> i test dymny

2.7 Zakres funkcjonalny AlphaPulse jako punkt odniesienia dalszych rozdziałów

Na poziomie operacyjnym framework obejmuje pełny przepływ pracy konkursowej:

1. pobranie danych (`train`, `validation`, `live`, `meta_model`)
2. uruchamianie eksperymentów opisanych YAML
3. automatyczną optymalizację hiperparametrów (HPO)
4. agentową pętlę badawczą (*AutoResearch*)
5. eksport predykcji do formatu submisji
6. inferencję na danych *live* i automatyczną submisję do API Numerai

W projekcie uwzględniono również:

- modułowy preprocessing (m.in. skalowanie, PCA, autoenkoder, kompresja, selekcja stabilności per-era)
- szeroką rodzinę modeli tabelarycznych (boosting, modele drzewiaste, liniowe, TabPFN/TabPFN3/TabICL, TabularDL)
- potoki `MultiHeadPipeline` i `MultiTargetPipeline` z neutralizacją cech
- strategie ensemble (`single`, `weighted`, `stacking`)
- diagnostykę eksperymentów w W&B (wykresy SHAP, ekspozycja cech, zbieżność HPO)
- panel EDA (Streamlit, 8 modułów analitycznych, interfejs PL/EN)
- standaryzację uruchomień przez zestaw skryptów CLI i targety `Makefile`

Ten zakres stanowi podstawę układu kolejnych rozdziałów: najpierw architektura, następnie implementacja i konfiguracja eksperymentów, a dalej ewaluacja wyników.

2.8 Mapowanie wyzwań domenowych na decyzje projektowe

Zależność między problemami domenowymi i mechanizmami systemu podsumowuje tabela 2.2.

Tabela 2.2: Wyzwanie finansowe → decyzja w AlphaPulse

Wyzwanie	Ryzyko	Decyzja projektowa
Struktura er	błędny split	walidacja era-aware, purge/embargo
Niestacjonarność	niestabilny sygnał	metryki per-era + Sharpe
Duża przestrzeń konfiguracji	koszt manualny	HPO + AutoResearch
Brak standaryzacji workflow	brak reprodukowalności	YAML + CLI + artefakty
Wymóg submisji	błąd formatu	dedykowany eksport <code>predict.pkl</code>

2.9 Wnioski

Z tego kontekstu wynika, że w Numerai trzeba łączyć trzy rzeczy naraz: poprawną metodologię, automatyzację eksperymentów i inżynierię zapewniającą reprodukowalność. Dlatego dalej traktujemy *AlphaPulse* nie jako pojedynczy model, tylko jako system badawczo-produkcyjny do iteracyjnego budowania i oceny pipeline'ów ML.

Rozdział 3

Architektura systemu AlphaPulse

W tym rozdziale opisujemy architekturę *AlphaPulse* jako konfigurowalny system AutoML dla danych finansowych Numerai: warstwy, przepływ danych, komponenty wykonawcze oraz to, jak układ wspiera reprodukowalność i iteracyjny rozwój eksperymentów.

3.1 Założenia architektoniczne

AlphaPulse opiera się na kilku założeniach:

- **Konfigurowalność:** eksperymenty są definiowane deklaratywnie, bez każdorazowej modyfikacji kodu źródłowego
- **Modularność:** preprocessing, modele i strategie ensemble są odseparowane i mogą być dowolnie komponowane
- **Reprodukowalność:** uruchomienia mają deterministyczny charakter przy ustalonej konfiguracji i ziarnie losowym
- **Rozszerzalność:** nowe komponenty mogą być dodawane bez naruszania istniejącego przepływu
- **Automatyzacja:** architektura wspiera zarówno klasyczne eksperymenty, jak i zautomatyzowane przeszukiwanie konfiguracji (HPO, AutoResearch)

3.2 Widok warstwowy systemu

Architekturę systemu można opisać jako układ pięciu współpracujących warstw.

3.2.1 Warstwa danych

Warstwa danych odpowiada za:

- pobranie i lokalne utrzymanie zbiorów Numerai
- ładowanie podziałów (np. train/validation)
- przygotowanie macierzy cech, celu i metadanych (era)
- kontrolę parametrów wejściowych (np. subsampling, feature set)

Ta warstwa dostarcza ustandaryzowany interfejs wejściowy dla całego pipeline'u.

3.2.2 Warstwa konfiguracji eksperymentu

Konfiguracja eksperymentu jest deklarowana w formacie YAML (schema-driven). Opisuje m.in.:

- źródło danych i parametry podziału
- zestawy cech i grupy cech
- listę preprocessingu
- typy modeli oraz ich hiperparametry
- strategię ensemble
- parametry treningu i ewaluacji

Dzięki temu logika eksperymentu jest jawna i łatwa do wersjonowania.

3.2.3 Warstwa pipeline'u modelowego

Warstwa pipeline'u realizuje sekwencję: `Data` → `Preprocessing` → `Model(e)` → `Ensemble` → `Prediction`. W ramach tej warstwy:

- preprocessory przekształcają cechy wejściowe
- modele bazowe uczą się predykcji celu
- ensemble agreguje sygnały modeli cząstkowych

Pipeline jest budowany dynamicznie na podstawie konfiguracji, co umożliwia szybkie porównywanie wielu wariantów architektury modelowej.

3.2.4 Warstwa optymalizacji i automatyzacji

System wspiera dwa poziomy automatyzacji:

- **HPO**: przeszukiwanie przestrzeni hiperparametrów i konfiguracji komponentów z użyciem Optuna TPE (flaga `--local`) lub Ray Tune (tryb domyślny rozproszony). Każda próba działa w izolowanym podprocesie; stan prób jest persystowany w SQLite (`TrialDB`), co umożliwia wznowienie przerwanej serii (`--resume`). Głównym celem optymalizacji w badaniu jest oficjalny `numera1_corr_sharpe`; raport obejmuje także średni CORR, maksymalne obsunięcie, udział dodatnich er oraz MMC na późniejszym przedziale czasowym

- **AutoResearch**: agentowa pętla badawcza, która proponuje mutacje konfiguracji (dodanie lub usunięcie modelu, zmianę preprocessingu, ensemble albo neutralizację) i ocenia ich wpływ na metryki

Wynikiem działania tej warstwy jest ranking prób, najlepsza konfiguracja (`best_config.json`), historia decyzji eksperymentalnych oraz diagnostyka w `Weights & Biases`.

3.2.5 Warstwa ewaluacji i diagnostyki

Warstwa ewaluacji obejmuje:

- **Backtester** i **EraSplitEvaluator** z opcjonalną neutralizacją cech (`FeatureNeutralizer`) przed obliczeniem metryk
- walidację krzyżową *PurgedEraCV* oraz walk-forward z *purge/embargo*
- raporty SHAP (`shap_report.py`) i ważności cech per-era
- logowanie wykresów diagnostycznych do W&B (`wandb_diagnostics.py`): korelacja per-era, ekspozycja cech, heatmapa korelacji ensemble, zbieżność HPO

3.2.6 Warstwa artefaktów i eksportu

Warstwa artefaktów odpowiada za:

- zapisy metryk i logów eksperymentów
- serializację najlepszych konfiguracji
- eksport predykcji do formatu wymaganej submisji (np. `predict.pkl`)

To domyka przepływ od eksperymentu badawczego do artefaktu konkursowego.

3.3 Przepływ wykonania end-to-end

Typowy scenariusz działania systemu obejmuje następujące etapy:

1. pobranie danych i przygotowanie lokalnego katalogu roboczego
2. uruchomienie eksperymentu z pliku YAML
3. trenowanie i ewaluacja modeli bazowych oraz ensemble
4. (opcjonalnie) uruchomienie HPO lub AutoResearch
5. wybór najlepszej konfiguracji i eksport predykcji

W ujęciu operacyjnym odpowiada temu zestaw dedykowanych skryptów CLI, które oddzielają definicję eksperymentu od mechaniki wykonania.

3.4 Komponenty funkcjonalne i ich odpowiedzialności

Architektura dzieli odpowiedzialności pomiędzy komponenty:

- **Data Loader**: odczyt danych i przygotowanie splitów
- **Experiment Schema**: walidacja konfiguracji wejściowej
- **Pipeline Builder**: konstrukcja grafu preprocessingu, modeli i ensemble
- **Trainer/Evaluator**: trening i obliczanie metryk jakości
- **HPO Engine**: zarządzanie próbami optymalizacyjnymi
- **AutoResearch Loop**: agentowe sterowanie kolejnymi eksperymentami
- **Exporter**: generowanie artefaktów konkursowych

Takie rozdzielenie upraszcza testowanie i ułatwia rozwój systemu.

3.5 Wzorce zapewniające jakość inżynierską

W *AlphaPulse* przyjęliśmy praktyki zwiększające niezawodność:

- **Separation of concerns** między konfiguracją, logiką uczenia i eksportem
- **CLI-first workflow** dla łatwej automatyzacji uruchomień
- **Jednoznaczne artefakty wyjściowe** dla porównywania eksperymentów
- **Integracja testów i narzędzi jakości kodu** dla stabilności rozwoju

W efekcie architektura nadaje się zarówno do badań iteracyjnych, jak i do utrzymania dłuższej linii rozwojowej projektu.

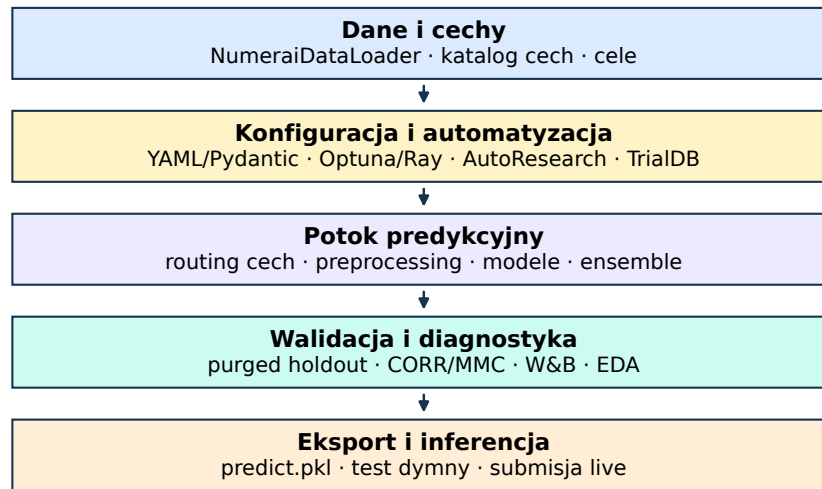
3.6 Ograniczenia architektury

Pomimo wysokiej elastyczności architektura ma ograniczenia:

- skuteczność zależy od jakości zdefiniowanej przestrzeni konfiguracji
- duża liczba wariantów zwiększa koszt obliczeniowy eksperymentów
- automatyzacja agentowa wymaga kontroli jakości proponowanych mutacji

Ograniczenia te są naturalne dla systemów AutoML i stanowią punkt wyjścia do dalszej optymalizacji.

3.7 Diagram architektury logicznej



Rysunek 3.1: Uproszczona architektura logiczna AlphaPulse. Diagram wygenerowano z aktualnej struktury implementacji; ograniczenie liczby komponentów zapewnia czytelność w druku.

Oprócz diagramu warstwowego, uproszczony przepływ end-to-end można przedstawić jako: Dane → YAML → Pipeline → HPO / AutoResearch → Artefakty / Export.

3.8 Kontrakty komponentów

Kontrakty komponentów określają minimalne wymagania wejścia i wyjścia dla kolejnych warstw systemu. Dzięki temu pipeline może być rozwijany modularnie: nowy loader, preprocessor, model lub strategia ensemble musi spełnić ten sam interfejs logiczny, bez konieczności zmiany pozostałych części frameworku.

Tabela 3.1: Podstawowe kontrakty komponentów systemu **AlphaPulse**

Komponent	Wejście	Wyjście
Loader	konfiguracja zbioru Numerai, wersja danych, lista cech	macierze X , y , kolumna era oraz metadane zbioru
Preprocessor	dane treningowe i parametry transformacji	przekształcone cechy oraz zapisany stan transformacji
Model	cechy, target, konfiguracja	wytrenowany estymator i
bazowy	hiperparametrów	predykcje walidacyjne
Ensemble	predykcje modeli bazowych i konfiguracja łączenia	finalna predykcja oraz metadane wag lub meta-modelu
Eksport	predykcje końcowe i identyfikatory przykładów	plik predict.pkl zgodny z wymaganiami Numerai

Konfiguracja eksperymentu pełni rolę wspólnego kontraktu sterującego. Wymagane sekcje YAML obejmują **data**, **features**, **preprocessing**, **models**, **ensemble_method**, **train** oraz **evaluation**. Każde uruchomienie zapisuje konfigurację, metryki, logi prób oraz, zależnie od trybu pracy, artefakty takie jak **best_config.json** i **predict.pkl**.

Architektura *AlphaPulse* jest sterowana konfiguracją i obejmuje cykl konkursowy: dane, eksperymenty, automatyzację i eksport predykcji. Modularność ma tu sens dopiero razem z reprodukowalnością; stąd kolejne rozdziały o implementacji i eksperymentach.

Rozdział 4

Implementacja systemu AlphaPulse

Tu przechodzimy do implementacji *AlphaPulse*: organizacji kodu, konfiguracji eksperymentów, CLI oraz przebiegu pipeline’u od danych do artefaktu submisyjnego.

4.1 Organizacja repozytorium

Implementacja projektu została podzielona na warstwy odpowiadające cyklowi pracy eksperymentalnej:

- `src/alphapulse/` - kod domenowy (pipeline, HPO, ewaluacja, `AutoResearch`, `features/`)
- `scripts/` - skrypty wykonawcze CLI dla najważniejszych scenariuszy
- `experiments/` - deklaratywne definicje eksperymentów (YAML)
- `eda/` - samodzielny panel Streamlit do analizy eksploracyjnej
- `tests/` - testy jednostkowe i integracyjne (w tym metryki MMC)
- `artifacts/` - wyniki uruchomień, konfiguracje i eksporty

Taki podział rozdziela logikę frameworku od warstwy operacyjnej (uruchamianie), co zwiększa czytelność i ułatwia utrzymanie projektu.

4.2 Warstwa uruchomieniowa (CLI)

4.2.1 Skrypty wykonawcze

System udostępnia zestaw skryptów CLI obsługujących pełny przepływ:

- `download_dataset.py` - pobieranie danych Numerai
- `run_experiment.py` - uruchamianie eksperymentów YAML (z opcją `--wandb-project`)
- `hpo_pipeline.py` - HPO (Optuna lokalnie lub Ray Tune)
- `autoresearch.py` - pętla agentowa AutoResearch

- `run_test_pipeline.py` - lekki test end-to-end
- `export_numerai_pickle.py`, `export_from_yaml.py` - eksport `predict.pkl`
- `live_inference.py` - predykcje na `live.parquet`
- `submit_predictions.py` - upload predykcji do API Numerai
- `make_feature_groups.py` - generowanie grup cech z `features.json`

Skrypty pełnią rolę cienkiej warstwy orkiestracji: pobierają argumenty, walidują wejście, inicjują komponenty frameworku i zapisują artefakty.

4.2.2 Konwencja parametrów wejściowych

Interfejs CLI wykorzystuje jednolitą konwencję argumentów:

- ścieżki danych i artefaktów (`--data-dir`, `--output-dir`)
- parametry skali eksperymentu (`--train-subsample`, liczba prób)
- wskazanie konfiguracji źródłowej (`--config`, `--best-config-path`)
- tryb lokalny lub rozproszony (zależnie od skryptu)

Ujednolicenie argumentów upraszcza automatyzację i integrację z narzędziami CI/CD.

4.3 Konfiguracja eksperymentów (YAML)

4.3.1 Schemat i walidacja

Eksperymenty definiowane są deklaratywnie. Schemat obejmuje sekcje:

- `data`
- `features`
- `preprocessing`
- `models`
- `ensemble_method/ensemble_params`
- `train`
- `evaluation`

Każde uruchomienie przechodzi walidację zgodności konfiguracji, co ogranicza liczbę błędów wykonania i gwarantuje spójność eksperymentów.

4.3.2 Konfigurowalność bez zmian kodu

Najważniejszą własnością implementacji jest możliwość modyfikacji eksperymentu poprzez edycję YAML:

- zmiana modelu i hiperparametrów
- podmiana preprocessingu
- przełączenie strategii ensemble
- zmiana metryki głównej ewaluacji

Pozwala to prowadzić szybkie iteracje badawcze przy zachowaniu pełnej ścieżki audytu zmian w repozytorium.

4.4 Implementacja warstwy danych

4.4.1 Loader danych Numerai

Warstwa danych dostarcza zunifikowany interfejs ładowania splitów oraz metadanych. Implementacja obejmuje:

- odczyt plików parquet i plików opisowych
- wybór zestawu cech i subsamplingu
- przygotowanie struktur `X`, `y`, `era`

Taki kontrakt danych jest wykorzystywany przez wszystkie wyższe warstwy pipeline'u.

4.4.2 Obsługa danych wejściowych pod eksperymenty

Loader obsługuje zarówno pełne zbiory, jak i tryby szybkich testów na podzbiórach danych. To umożliwia:

- szybkie iteracje deweloperskie
- tańsze testy poprawności potoku
- stopniowe skalowanie kosztownych eksperymentów

4.5 Implementacja pipeline’u modelowego

4.5.1 Preprocessing

Implementacja preprocessingu wykorzystuje listę kroków konfigurowanych w YAML. Wspierane są m.in.:

- skalowanie (*StandardScaler*, *RobustScaler*)
- redukcja wymiarowości (*PCA*, *TruncatedSVD*)
- selekcja cech i transformacje regularizujące

Kroki preprocessingu są wykonywane deterministycznie i mogą być łączone w sekwencje.

4.5.2 Modele bazowe

Warstwa modeli została zaprojektowana jako zestaw wymiennych adapterów (spójny interfejs *fit/predict*) dla różnych rodzin:

- gradient boosting
- modele drzewiaste
- modele liniowe
- wybrane modele fundamentowe dla danych tabelarycznych

Standaryzacja interfejsu upraszcza porównywanie modeli i ich integrację z ensemble.

4.5.3 Strategie ensemble

Implementacja obsługuje trzy podstawowe tryby:

- **single** - pojedynczy model
- **weighted** - ważona agregacja predykcji
- **stacking** - meta-model uczony na predykcjach bazowych [14]

Strategia ensemble jest traktowana jako pełnoprawny element konfiguracji eksperymentu.

4.6 Implementacja HPO

4.6.1 Silniki optymalizacji

HPO realizowane jest przez moduł `src/alphapulse/hpo/` z dwoma trybami uruchomienia:

- **Tryb lokalny** (`--local`): Optuna TPE [8] (Bayesowski sampler) z izolacją każdej próby w osobnym podprocesie; baza `optuna.db` i `trials.db` (TrialDB)
- **Tryb rozproszony**: Ray Tune [9] z callbackiem W&B; wymaga zależności `alphapulse[hpo]`

Wspólne elementy obu trybów:

- definicja przestrzeni przeszukiwania (`search_space.py`, `registry.py`)
- funkcja celu (`objective.py`) z oceną era-holdout lub walk-forward (3 foldy)
- routing cech (`feature_routing.py`) i strategię wielocelowe (`target_strategy.py`)
- limit czasu próby (`--trial-timeout`, domyślnie 3600 s) i budżet godzinowy (`--max-hours`)
- eksport najlepszej konfiguracji (`hpo/export.py`)

4.6.2 Śledzenie wyników prób

Każda próba HPO generuje komplet informacji eksperymentalnych:

- parametry wejściowe i hash konfiguracji
- oficjalne diagnostyki `numera_i_corr_*` i `numera_i_mmc_*` dla holdoutu oraz późniejszego przedziału czasowego
- wykresy diagnostyczne w W&B (ważność cech, ekspozycja, korelacja per-era)
- metadane wykonania (czas, typ modelu, ewentualny błąd)

Po zakończeniu sweepu logowany jest run `best-trial-diagnostics` oraz wykresy `search-convergence` i `hpo-summary` (scatter: corr Sharpe vs. MMC Sharpe vs. czas).

4.7 Implementacja AutoResearch

4.7.1 Pętla agentowa

AutoResearch rozszerza klasyczny HPO o mechanizm agentowy:

1. analiza dotychczasowych wyników
2. propozycja mutacji konfiguracji
3. uruchomienie kolejnej próby
4. aktualizacja stanu badania

Stan eksperymentu i decyzje agenta są zapisywane w artefaktach, co pozwala audytować przebieg procesu badawczego.

4.7.2 Rola AutoResearch w implementacji

Mechanizm ten nie zastępuje pełni pracy badacza, lecz automatyzuje najbardziej powtarzalny etap: generowanie kolejnych hipotez konfiguracyjnych i ich testowanie. Wpływ tej automatyzacji na czas pracy wymaga kontrolowanego pomiaru i nie jest rozstrzygany na podstawie samej implementacji.

4.8 Eksport predykcji i artefakty końcowe

Kończącym etapem implementacji jest produkcja artefaktu submisyjnego:

- odtworzenie najlepszego pipeline’u z `best_config.json` lub YAML
- trening na wskazanym zbiorze z zachowaniem kontraktu schematu cech
- wygenerowanie predykcji z normalizacją rangową per-era
- serializacja do `predict.pkl` z nazwą kanoniczną `<TIMESTAMP>_<ARCH>_<TARGET>_<CONFIG_HASH>_predict.pkl` i symlinkiem `latest_predict.pkl`
- zapis `*_provenance.json`: rozwiązana konfiguracja, snapshot zależności (`uvexport`) i hash commita Git
- test dymny: czysty podproces ładuje pickle i wykonuje forward pass na syntetycznej ramce danych

4.8.1 Workflow produkcyjny (inferencja i submisja)

Tygodniowy cykl turniejowy realizowany jest trzema krokami:

1. pobranie świeżych danych (`live.parquet`)
2. `live_inference.py` - predykcje z walidacją formatu (`--validate`)

3. `submit_predictions.py` - upload CSV (`id`, `prediction`) do API Numerai

Skrypty wymagają kluczy `NUMERAI_PUBLIC_API_KEY` i `NUMERAI_PRIVATE_API_KEY` (lokalny plik `.env` lub zmienne środowiskowe).

4.9 Zapewnienie jakości kodu

W implementacji używamy standardowych mechanizmów jakości:

- linting i formatowanie (`ruff`, `make fmt`, `make lint`)
- statyczne sprawdzanie typów (`mypy`, `make types`)
- testy automatyczne z pokryciem (`pytest`, `make test`)
- wykrywanie martwego kodu (`vulture`, `make deadcode`)
- hooki pre-commit i target `make check` (`lint` + `types` + `test` + `deadcode`)
- osobna ścieżka jakości dla modułu EDA (`make eda-lint`)

To ogranicza regresje i zwiększa stabilność frameworku w kolejnych iteracjach.

4.10 Przykładowa konfiguracja referencyjna

```
version: "1"
data:
  data_dir: data/v5.3
  train_subsample: 0.10
  target_col: target_ender_60
  seed: 20260824
models:
  - type: XGBoost
    params:
      max_depth: 3
      learning_rate: 0.05
      tree_method: hist
ensemble_method: single
evaluation:
  primary_metric: numerai_corr_sharpe
```

4.11 Przestrzeń przeszukiwania HPO

Przykładowe parametry optymalizowane przez HPO (Optuna/Ray Tune):

- typ modelu (XGBoost/LightGBM/CatBoost/Ridge/...)
- hiperparametry modelu (`max_depth`, `learning_rate`, `n_estimators`)
- wybór preprocessingu i jego parametrów
- metoda ensemble i parametry meta-modelu

W badaniu końcowym przestrzeń ograniczono do jednego modelu z puli XGBoost, LightGBM, CatBoost, TabPFN i TabICL. Po 20 próbach inicjalnych sampler TPE korzystał z dotychczasowej historii, a każda konfiguracja była oceniana w tym samym protokole czasowym. Szczegółowy budżet opisano w rozdziale 5.

Rozdział 5

Metodyka badań

Rozdział opisuje faktycznie wykonane serie, reguły włączenia obserwacji do analizy oraz sposób wyboru kandydatów. Chodzi o rozdzielenie dwóch rodzajów dowodów: weryfikacji skalowalności całego systemu i porównania jakości konfiguracji w jednolitym protokole Numerai v5.3.

5.1 Pytania eksperymentalne

Analiza odpowiada na cztery pytania:

1. Czy **AlphaPulse** wykonuje kompletny potok na pełnym zbiorze danych i zapisuje artefakty umożliwiające odtworzenie uruchomienia?
2. Które rodziny modeli osiągają najwyższy oficjalny CORR w ustalonym budżecie i podziale czasowym?
3. Jak zgodne są rankingi na purged holdoutcie i późniejszym przedziale walidacyjnym oraz które konfiguracje tworzą kompromis CORR-MMC?
4. Jaki koszt wykonania i poziom kompletności prób uzyskano w serii HPO?

Porównania rodzin mają charakter opisowy. TPE wybiera konfiguracje adaptacyjnie, dlatego liczebności grup nie są równe, a obserwowane różnice nie są traktowane jako kontrolowana ablacja ani dowód przyczynowy.

5.2 Dwie serie o rozłącznych rolach

Materiał empiryczny obejmuje benchmark pełnoskalowy i końcową serię HPO (tabela 5.1). Różne wersje danych i cele wykluczają wspólny ranking jakości. Pierwsza seria odpowiada na pytanie o wykonalność i czas pełnej ścieżki, druga - o jakość konfiguracji według oficjalnych metryk.

Benchmark wykorzystał wszystkie 574 ery uczące. Potok **StandardScaler** oraz **EraEnsemble** z dziesięcioma modelami **XGBoost** przetworzył następnie 3 895 460 wierszy walidacyjnych w 1888,6 s, czyli 31,5 min. Wynik ten dokumentuje możliwość pracy na pełnej skali; jego miary jakości nie są przenoszone do rankingu v5.3.

Tabela 5.1: Serie wykorzystane w badaniu i ich role

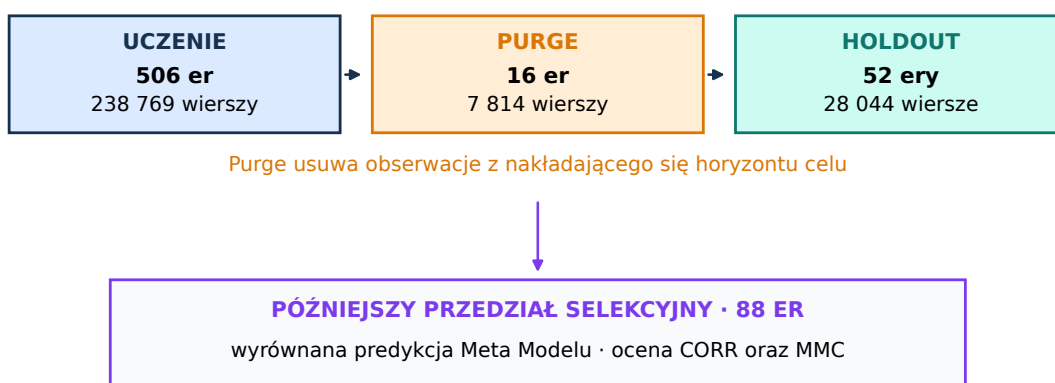
Seria	Dane	Cel	Zakres train	Rola
Benchmark pełnoskalowy	v5.2	<code>target</code>	2 746 268 wierszy, 780 cech	skalowalność i kompletność przepływu
Końcowe HPO	v5.3	<code>target_ender_60</code>	274 627 wierszy, 780 cech	porównanie oficjalnego CORR i MMC

5.3 Dane i podział czasowy badania końcowego

Końcowe HPO korzysta z deterministycznej, dziesięcioprocentowej próby zbioru `train.parquet`: 274 627 wierszy, 574 ery i 780 cech bazowych. Losowanie ma stałe ziarno 20260824, a integralność wejścia jest utrwalona skrótami SHA-256 w pliku `protocol.json`. Każda próba przewiduje `target_ender_60`.

Ostatnie 52 ery próbki tworzą holdout. Szesnaście bezpośrednio poprzedzających er zostaje usuniętych z uczenia ze względu na nakładający się horyzont celu. W efekcie część ucząca obejmuje 506 er i 238 769 wierszy, strefa separacji 7814 wierszy, a holdout 28 044 wiersze. Drugim punktem oceny jest późniejszy fragment `validation.parquet`: 88 er, dla których dostępna jest wyrównana predykcja Meta Modelu. Umożliwia to obliczenie zarówno CORR, jak i MMC. Fragment ten nie steruje samplerem TPE, lecz uczestniczy w wyborze kandydatów końcowych, dlatego w dalszej części jest nazywany późniejszym przedziałem selekcyjnym, a nie zbiorem testowym.

Próba ucząca v5.3: 274 627 wierszy, 574 ery, 780 cech bazowych



Rysunek 5.1: Podział czasowy badania końcowego. Strefa 16 er oddziela uczenie od holdoutu; późniejszy przedział 88 er służy do analizy transferu, wyboru kandydatów oraz oceny MMC.

5.4 Protokół HPO

Parametry protokołu zostały zapisane przed serią i nie zmieniały się między próbami. Najważniejsze ustawienia przedstawia tabela 5.2.

Tabela 5.2: Protokół końcowej serii HPO

Element	Ustawienie
Wersja danych i cel	Numerai v5.3, <code>target_ender_60</code>
Próba i ziarno	10%, 274 627 wierszy, seed 20260824
Podział	506 er train, 16 er purge, 52 ery holdout
Późniejszy przedział	88 er z predykcją Meta Modelu
Funkcja celu	oficjalny <code>numerai_corr_sharpe</code> na holdoucie
Sampler	Optuna TPE, 20 prób inicjalnych
Przestrzeń modeli	XGBoost, LightGBM, CatBoost, TabPFN, TabICL
Złożoność próby	maksymalnie jeden model, routing grup cech
Limity	200 prób lub 11,5 h; 1800 s na próbę
Implementacja metryk	<code>numerai-tools==0.6.0</code>
Wersja kodu	commit 1a86893, protokół <code>corrected-hpo-2026-08-v5</code>

Do analizy włączono wyłącznie rekordy ze statusem `completed`, kompletną konfiguracją i skończonym wektorem trzech miar: CORR Sharpe na holdoucie, CORR Sharpe na późniejszym przedziale oraz MMC Sharpe na tym przedziale. Wcześniejsze uruchomienia kontrolne z identycznym ziarnem i konfiguracją nie są liczone ponownie.

5.5 Przestrzeń konfiguracji

Jedna próba wybiera rodzinę modelu, grupy cech, skalowanie, opcjonalną selekcję wariancji, parametry regularizacji, liczbę iteracji i stopień neutralizacji względem cech. Routing łączy zestawy bazowe (`small`, `medium`) z grupami tematycznymi z katalogu Numerai. Liczba wejściowych cech różni się zatem między konfiguracjami.

Dla modeli boostingowych optymalizowane są między innymi: tempo uczenia, głębokość lub liczba liści, minimalny rozmiar liścia, subsampling kolumn i wierszy oraz regularyzacja. TabPFN i TabICL działają w profilach pamięciowych ograniczających liczbę przykładów oraz wymiar po redukcji. Ich wyniki opisują zachowanie w tym konkretnym budżecie, a nie ogólną przewagę jednej architektury nad drugą.

5.6 Metryki i reguła selekcji

Główną metryką jest oficjalny CORR Sharpe z równania (2.5) na 52-erowym holdoutcie. Dla obu przedziałów raportowane są średni CORR, odchylenie, udział dodatnich er i maksymalne obsunięcie. Na 88-erowym późniejszym przedziale obliczany jest dodatkowo oficjalny MMC Sharpe.

Nie tworzą syntetycznej funkcji łączącej CORR i MMC. Zamiast tego wyróżniono:

1. lidera CORR Sharpe na holdoutcie
2. lidera CORR Sharpe na późniejszym przedziale
3. lidera MMC Sharpe na późniejszym przedziale
4. zbiór Pareto względem CORR i MMC na późniejszym przedziale

Taka reguła zachowuje interpretowalność kompromisu między siłą sygnału i jego wkładem niezależnym od Meta Modelu. Jednocześnie wybór maksimum spośród 27 prób powoduje błąd selekcyjny, więc wartości kandydatów opisują badaną serię, a nie nieobciążone oczekiwanie wyniku przyszłego [24, 23].

5.7 Metody analizy

Dla każdej kompletnej próby zestawiono metryki, czas wykonania, rodzinę modelu i liczbę trasowanych cech. Zgodność dwóch przedziałów czasowych oceniono korelacją Pearsona oraz korelacją rang Spearmana. Ze względu na małą liczbę konfiguracji i adaptacyjny sposób ich doboru współczynniki te są statystykami opisowymi; nie raportuje się dla nich testów istotności ani przedziałów ufności. Rodziny modeli porównano za pomocą median, wykresów pudełkowych i wszystkich punktów, zawsze z podaniem liczebności.

Front Pareto wyznaczono przez dominację dwukryterialną: konfiguracja należy do frontu, jeżeli nie istnieje inna próba o co najmniej takim samym CORR Sharpe i MMC Sharpe oraz wyniku ściśle lepszym w co najmniej jednym kryterium. Przebieg HPO analizowano przez maksimum narastające, rozdzielając 20 prób inicjalnych i fazę adaptacji TPE.

5.8 Reprodukowalność

Kompletna proveniencja końcowej serii znajduje się w plikach `protocol.json`, `trials.db` i `optuna.db`. Protokół zawiera wersję Pythona, commit, skróty kodu i danych, seed, liczbę próbkowanych wierszy, cel oraz parametry oceny. Konfiguracja każdej próby i jej metryki są przechowywane w pojedynczym rekordzie SQLite, a przebieg jest równolegle rejestrowany w W&B. Zakres proveniencji odpowiada zaleceniom raportowania kodu, danych, zależności i warunków wykonania w badaniach uczenia maszynowego [21].

Odtworzenie badania w środowisku projektu wymaga następujących poleceń:

```
uv sync --extra dev --extra hpo --extra foundation
uv run python scripts/download_dataset.py --config.dataset-version v5.3
    --config.output-dir data
uv run python scripts/hpo_pipeline.py --data-dir data/v5.3 --target-col
    target_ender_60 --train-subsample 0.10 --seed 20260824 --num-trials 200
    --output-dir artifacts/hpo_v53_final --local --objective numerai_corr_sharpe
    --purge-eras 16 --max-hours 11.5 --trial-timeout 1800 --sampler tpe
    --n-startup-trials 20 --max-models 1 --gpu --wandb-project
    alphapulse-v53-final --no-wandb-diagnostics
```

5.9 Zagrożenia dla trafności

Badanie końcowe obejmuje jedno ziarno i 27 kompletnych konfiguracji. Adaptacyjny dobór prób powoduje nierówne liczebności rodzin, dlatego agregaty mają charakter opisowy, a nie testowy. HPO korzysta z 10% danych uczących; wyników nie ekstrapoluje się na pełny trening. Benchmark v5.2 potwierdza skalowalność, lecz inny cel i wersja danych wykluczają porównanie jakości.

Holdout był wielokrotnie wykorzystywany przez funkcję celu. Późniejszy przedział 88 er nie sterował samplerem, ale służył do porównania prób i wyboru kandydatów. Nie jest więc nietkniętym zbiorem testowym, a maksimum może być optymistycznie obciążone. Oba przedziały reprezentują po jednym zakresie rynkowym. Wdrożenie produkcyjne powinno poprzedzać ponowne uczenie wybranej receptury na pełnych danych oraz monitoring wielu kolejnych rund live.

Badanie nie zawiera kontrolowanego baseline’u jakości ani ablacji komponentów. Benchmark pełnoskalowy używa innej wersji danych i celu, a próby HPO różnią się jednocześnie rodziną modelu, routinguem cech, preprocessingiem i neutralizacją. W tej serii widać, że system działa i dostajemy opisowy ranking w zadanym budżecie; różnicy jakości nie da się jednak przypisać pojedynczemu mechanizmowi.

Rozdział 6

Wyniki eksperymentów

Poniżej wyniki benchmarku pełnoskalowego i końcowej serii HPO. Rankingi jakości biorą się z jednego protokołu v5.3 i oficjalnych metryk Numerai; starszych uruchomień kontrolnych tu nie mieszamy.

6.1 Weryfikacja pełnej skali

Benchmark pełnoskalowy objął 2 746 268 wierszy uczących, 780 cech i 574 ery. Potok z dziesięcioma partycjami XGBoost wykonał trening, predykcję dla 3 895 460 wierszy walidacyjnych, ocenę i zapis diagnostyki w 1888,6 s (31,5 min). Utworzone zostały konfiguracja rozwiązana, hash konfiguracji i pełny zapis przebiegu. Rezultat potwierdza, że ta sama architektura obsługuje nie tylko próby HPO, lecz także trening na całym dostępnym zbiorze.

Tabela 6.1: Wynik operacyjny benchmarku pełnoskalowego

Wielkość	Wartość
Wiersze train / validation	2 746 268 / 3 895 460
Liczba cech / er train	780 / 574
Modele składowe	10 × XGBoost
Czas całkowity	1888,6 s (31,5 min)

6.2 Kompletność końcowej serii HPO

Seria została zamknięta po zapisaniu 32 prób. Kryteria kompletności spełniło 27 konfiguracji (84,4% całej puli); cztery rekordy nie zawierały wyniku, a jeden pozostał niedokończony. Analiza ilościowa obejmuje wyłącznie 27 kompletnych rekordów.

Łączny czas ukończonych prób wyniósł 1,44 h, a mediana czasu pojedynczej konfiguracji 127,2 s. Są to czasy prób modelowych, bez jednorazowego pobrania danych i przygotowania środowiska.

Tabela 6.2: Kompletność końcowej serii HPO

Status	Liczba	Udział
Kompletna próba	27	84,4%
Bez kompletnego wyniku	4	12,5%
Niedokończona	1	3,1%
Razem	32	100,0%

6.3 Rozkład oficjalnych metryk

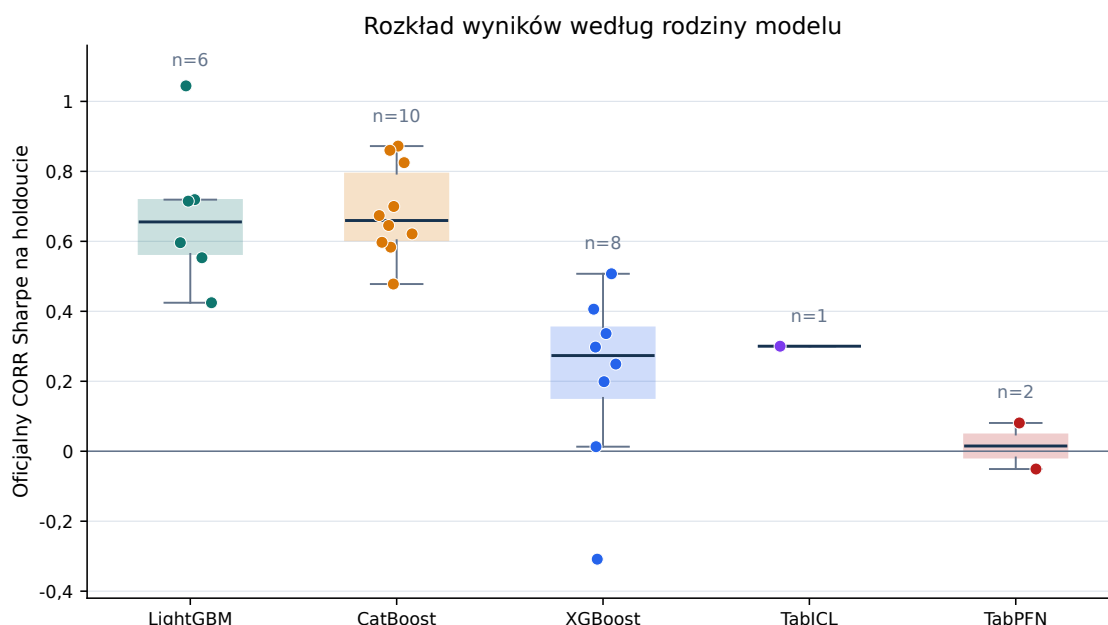
Tabela 6.3 syntetyzuje wszystkie kompletne konfiguracje. Na holdoucie dodatni CORR Sharpe uzyskało 25 z 27 prób. Na późniejszym przedziale selekcyjnym dodatni CORR Sharpe osiągnęły 24 próby, natomiast dodatni MMC Sharpe 13 prób. Różnica pokazuje, że sama korelacja z celem nie gwarantuje unikalności sygnału względem Meta Modelu.

Tabela 6.3: Podsumowanie oficjalnych metryk dla 27 kompletnych prób

Metryka	Średnia	Mediana	Wynik dodatni
CORR Sharpe, holdout (52 ery)	0,479	0,553	25/27
CORR Sharpe, późniejszy (88 er)	0,128	0,128	24/27
MMC Sharpe, późniejszy (88 er)	0,0247	-0,0098	13/27

6.4 Rodziny modeli w ustalonym budżecie

Najwyższe mediany CORR Sharpe na holdoucie uzyskały CatBoost (0,659; $n = 10$) i LightGBM (0,655; $n = 6$). Mediana XGBoost wyniosła 0,273 ($n = 8$), TabICL 0,300 ($n = 1$), a TabPFN 0,015 ($n = 2$). Wszystkie obserwacje i rozrzut przedstawia rysunek 6.1.

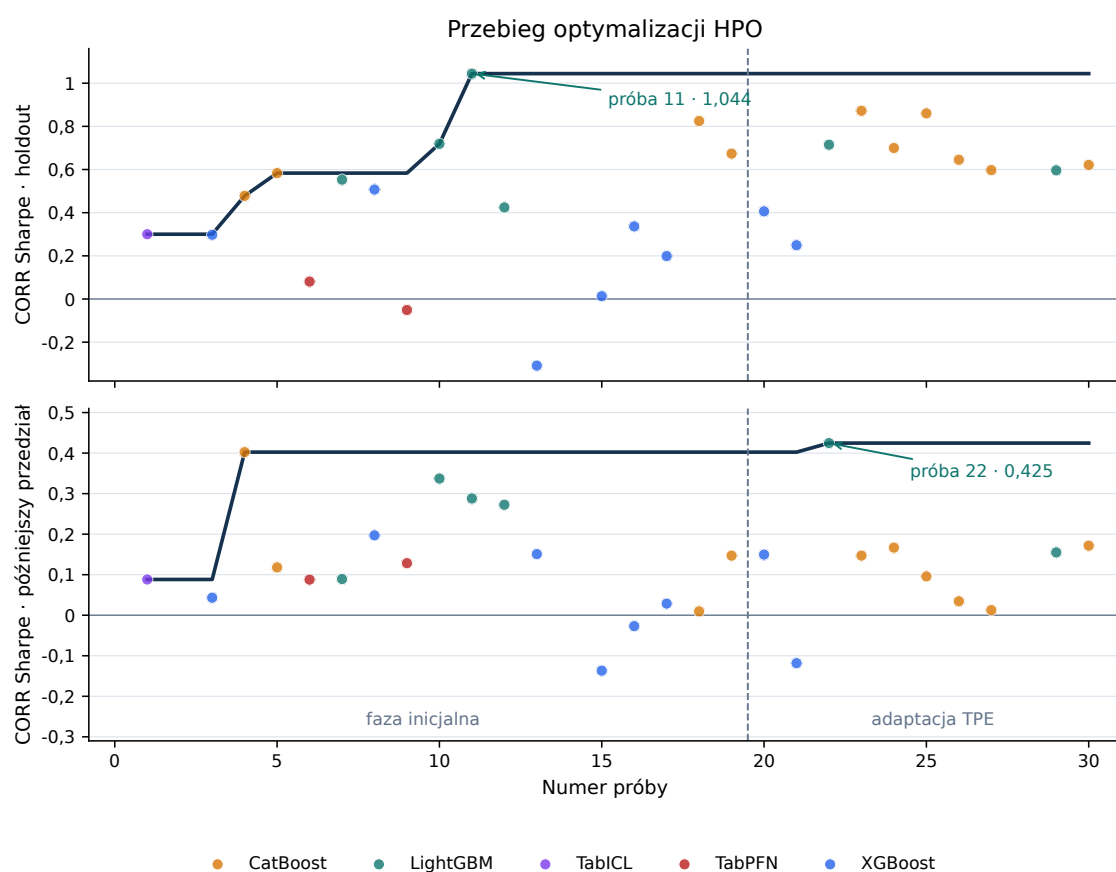


Rysunek 6.1: Oficjalny CORR Sharpe na holdoucie według rodziny modelu. Punkty oznaczają kompletne próby, a licznosci podano nad grupami.

Na późniejszym przedziale mediana LightGBM wyniosła 0,280, CatBoost 0,133, XGBoost 0,036, TabPFN 0,108, a TabICL 0,088. Wynik jest opisem badanego budżetu, nie rankingiem architektur w ogólności: TPE przydzielił rodzinom różne liczby prób, a modele fundamentowe korzystały z limitów liczby przykładów i redukcji wymiaru.

6.5 Przebieg optymalizacji

Pierwsze 20 konfiguracji stanowiło fazę inicjalną TPE. Lider holdoutu pojawił się już w próbie 11. Po przejściu do fazy adaptacyjnej poprawił się natomiast najlepszy wynik na późniejszym przedziale: próba 22 podniosła maksimum CORR Sharpe z 0,402 do 0,425. W kolejnych próbach maksimum nie uległo zmianie (rysunek 6.2).

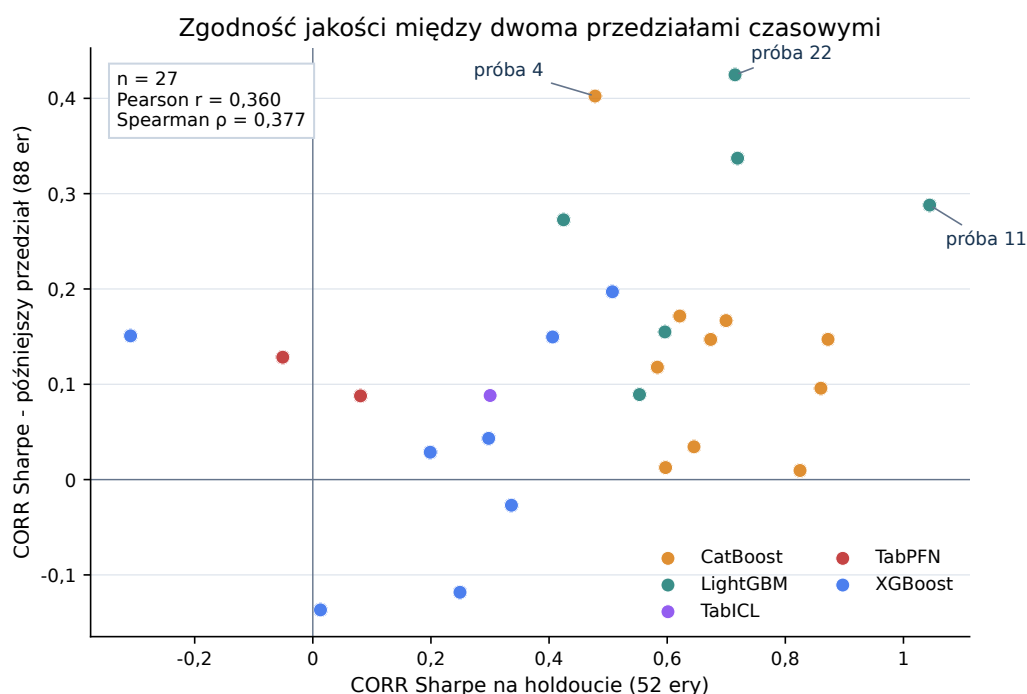


Rysunek 6.2: Wyniki prób i maksimum narastające dla holdoutu oraz późniejszego przedziału. Linia przerywana oddziela fazę inicjalną od adaptacji TPE.

W fazie adaptacyjnej pojawił się kandydat o najlepszym wyniku na późniejszym przedziale, mimo że sampler optymalizował holdout. Obserwacja ta opisuje przebieg serii, ale nie dowodzi przewagi TPE nad losowym próbkowaniem, ponieważ nie wykonano równoległego eksperymentu kontrolnego o tym samym budżecie.

6.6 Transfer między przedziałami czasowymi

Korelacja Pearsona między CORR Sharpe na holdoucie i późniejszym przedziale wyniosła $r = 0,360$, a korelacja rang Spearmana $\rho = 0,377$. Związek jest dodatni, lecz umiarkowany: wysoka pozycja na 52-erowym holdoucie tylko częściowo współwystępuje z dobrym wynikiem później i nie zachowuje rankingu konfiguracji (rysunek 6.3).



Rysunek 6.3: Zgodność CORR Sharpe między holdoutem i późniejszym przedziałem dla 27 kompletnych konfiguracji.

Próba 11 maksymalizuje holdout, lecz na późniejszym przedziale ustępuje próbom 22, 4 i 10. To najważniejszy argument za dwustopniową selekcją: holdout steruje HPO, natomiast późniejszy przedział rozstrzyga o przenoszeniu sygnału w czasie.

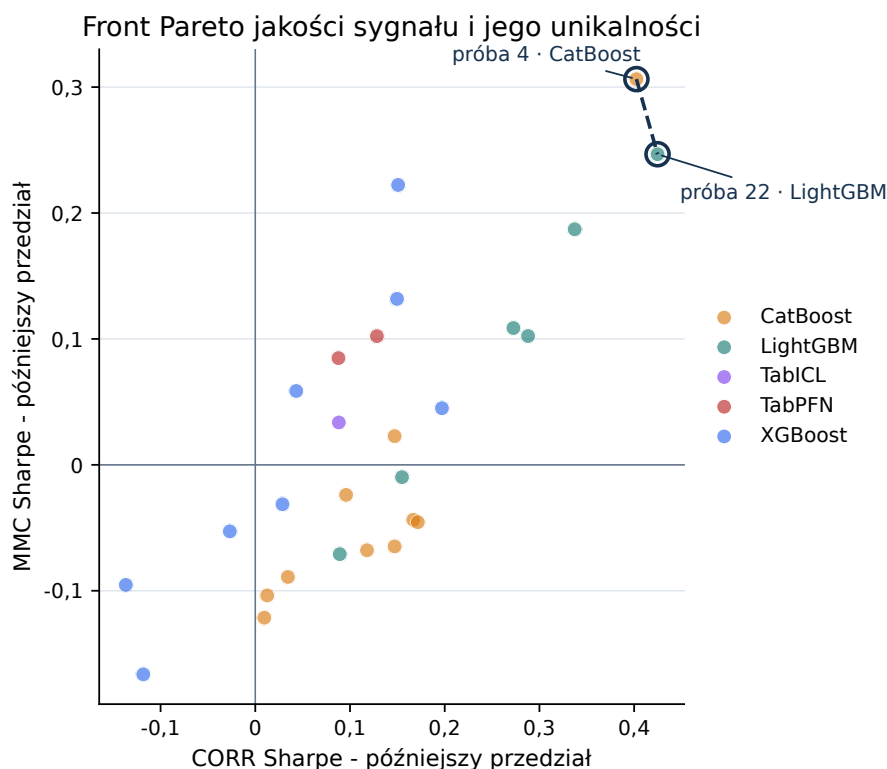
6.7 Kandydaci końcowi

Tabela 6.4 zestawia pięć reprezentatywnych konfiguracji. Próba 11 jest liderem holdoutu, próba 22 liderem CORR na późniejszym przedziale, a próba 4 liderem MMC. Próby 10 i 8 pokazują mocne alternatywy z rodzin LightGBM i XGBoost.

Tabela 6.4: Oficjalne metryki końcowych kandydatów; HO - holdout, PP - późniejszy przedział

Próba	Model	HO CORR		PP CORR		PP MMC	
		średnia	Sharpe	średnia	Sharpe	średnia	Sharpe
22	LightGBM	0,03032	0,7149	0,01679	0,4247	0,00802	0,2467
4	CatBoost	0,02161	0,4779	0,01483	0,4024	0,01031	0,3063
10	LightGBM	0,02743	0,7191	0,01450	0,3372	0,00716	0,1871
11	LightGBM	0,03742	1,0443	0,01038	0,2880	0,00310	0,1023
8	XGBoost	0,02047	0,5072	0,00705	0,1971	0,00142	0,0450

W przestrzeni CORR-MMC późniejszego przedziału tylko próby 22 i 4 nie są zdominowane (rysunek 6.4). Próba 22 maksymalizuje jakość sygnału, a próba 4 jego unikalność. Nie istnieje więc jeden zwycięzca niezależny od celu wdrożeniowego.



Rysunek 6.4: Front Pareto na późniejszym przedziale. Okręgi wyróżniają próby niezdominowane względem CORR Sharpe i MMC Sharpe.

Do treningu nastawionego na stabilny CORR wybrano próbę 22. Jest to LightGBM z RobustScalerem, selekcją wariancji, 989 cechami z grup `medium+agility+midnight`, 200 rundami, tempem uczenia 0,0200, 31 liśćmi i neutralizacją cech 0,2373. Jej CORR Sharpe wynosi 0,7149 na holdoucie i 0,4247 na późniejszym przedziale, przy dodatnim MMC Sharpe 0,2467.

Próba 4 jest alternatywą nastawioną na MMC. Wykorzystuje CatBoost, StandardScaler, 970 cech z sześciu grup, głębokość 5, tempo uczenia 0,0314 i neutralizację 0,4947. Osiąga CORR Sharpe 0,4024 oraz MMC Sharpe 0,3063 na późniejszym przedziale. Próba 11 pozostaje górną granicą jakości na holdoucie: CORR Sharpe 1,044, średni CORR 0,0374, maksymalne obsunięcie 0,0396 i 84,6% dodatnich er.

6.8 Synteza wyników

Z tej serii wnosimy trzy konkretne rzeczy. Pełny potok ogarnia miliony wierszy i zostawia odtwarzalne artefakty (poniżej 32 min w badanym benchmarku). Przy naszym budżecie najwyższe mediany holdoutu miały LightGBM i CatBoost. Późniejsze okno przestawiło ranking i zostawiło dwa praktyczne profile: próbę 22 pod CORR oraz próbę 4 pod MMC. To pomiary z tej serii, bez ekstrapolacji na pełny trening. Skoro okno brało udział w wyborze kandydatów, liczb nie traktujemy jako czystej prognozy przyszłego wyniku.

Rozdział 7

Dyskusja i zakończenie

7.1 Realizacja celu pracy

Cel - konfigurowalne AutoML pod potoki Numerai - domknęliśmy na trzech poziomach: architektura, kod i seria empiryczna. AlphaPulse spina pobieranie danych, walidowany YAML, routing cech, modele, HPO, ocenę czasową, proveniencję i eksport funkcji predykcyjnej. Pełnoskalowy benchmark pokazał, że przepływ w ogóle przechodzi na milionach wierszy; seria v5.3 - że da się porównywać różne konfiguracje jednym, domenowo poprawnym protokołem.

Sam estymator to za mało. Liczy się kontrolowane laboratorium: YAML mówi, co badacz chce zrobić, runtime trzyma ten sam split i scoring, a baza prób spina wynik z kodem, danymi i ziarnem. To klasyczny AutoML plus twarde wymagania finansowe [10, 8, 9].

7.2 Interpretacja wyników modelowych

LightGBM i CatBoost miały najwyższe mediany CORR Sharpe na holdoutcie. Pasuje to do tego, co wiemy o boostingu na tabelach [12, 13, 17]. Nie znaczy jednak, że zawsze wygrywają: sampler był adaptacyjny, a modele fundamentowe dostały ostrzejsze limity pamięci. W naszym budżecie i dla `target_ender_60` najlepsze konfiguracje wyszły z dwóch rodzin boostingowych - i na tym kończymy interpretację.

Umiarkowana korelacja rankingów holdout $\leftrightarrow$ późniejszy przedział podkreśla rolę separacji czasowej. Przy słabym sygnale łatwo „przykleić się” do lokalnej specyfiki jednego okna. Purging tnie przeciek z horyzontu celu, ale nie kasuje niestacjonarności [7, 22]. Dlatego 88 er służy do czego innego niż holdout HPO: nie kręci samplerem, tylko sprawdza transfer receptur. Skoro jednak wybieraliśmy na nich próby 22 i 4, to nie jest czysty test końcowy [24, 23].

Najbardziej użyteczny wynik to front Pareto. Próba 22 ciągnie CORR na późniejszym oknie przy dodatnim MMC; próba 4 odwrotnie - mocniejsze MMC przy nieco słabszym CORR. Dwa sensowne profile: stabilna zgodność z celem versus większy wkład niezależny od Meta Modelu [3, 4]. Lepiej trzymać obu kandyda-

tów niż sklejać metryki w jeden wskaźnik. Pamiętamy też, że liczby są warunkowe względem tej selekcji.

7.3 Odpowiedzi na pytania eksperymentalne

1. **Skalowalność i reprodukowalność.** Pełnoskalowy potok przetworzył 2,75 mln wierszy uczących i 3,90 mln wierszy walidacyjnych w 31,5 min. Każde uruchomienie zapisuje konfigurację, hash, metryki i artefakty, a końcowe HPO dodatkowo utrwała skróty danych oraz wersję kodu
2. **Rodziny modeli.** W badanym budżecie najwyższe mediany holdoutu uzyskały CatBoost i LightGBM. Wynik ma charakter opisowy, ponieważ HPO nie jest zrównoważoną ablacją
3. **Transfer i kompromis CORR-MMC.** Zgodność dwóch przedziałów jest dodatnia, ale umiarkowana. Próby 22 i 4 tworzą front Pareto na późniejszym przedziale i reprezentują odpowiednio profil CORR oraz MMC
4. **Koszt i kompletność procesu.** Analiza obejmuje 27 kompletnych prób z medianą 127,2 s. Baza TrialDB zachowała 84,4% uruchomionej puli jako kompletne rekordy możliwe do bezpośredniego porównania

Te odpowiedzi siedzą w artefaktach. Nie twierdzimy, że system skraca pracę człowieka ani że AutoResearch wygrywa z TPE - na to trzeba osobnego eksperymentu.

7.4 Znaczenie inżynierskie

Wymagania domenowe da się zamknąć w kontraktach kodu. Era steruje podziałem, nie jest zwykłą cechą; horyzont celu wyznacza separację; oficjalne metryki siedzą w ewaluatorze; format submisji sprawdzamy w czystym podprocesie. Ryzyka metodologiczne łapie więc architektura, a nie tylko dyscyplina prowadzącego.

Warstwa artefaktów robi resztę. `protocol.json` trzyma wejście i reguły, `trials.db` - stan prób, W&B - krzywe i diagnostykę. Razem zostaje ślad audytowy pod pracę naukową i pod utrzymanie modelu. Eksport `predict.pkl` zamyka lukę między labem a inferencją live.

7.5 Ograniczenia

Budżet końcowej serii jest wąski: jedno ziarno, 27 kompletnych konfiguracji, dziesięcioprocentowa próba train. Nie policzymy wariancji między seedami ani przedziałów ufności dla rankingów. TPE dodatkowo utrudnia przyczynowe porównanie rodzin.

Holdout wchodzi do funkcji celu, więc jego maksimum jest obciążone wielokrotnym porównaniem. Późniejsze 88 er nie kręci HPO, ale służyło do rankingów 27 prób i wyboru kandydatów - odcina zależność od holdoutu, ale nie daje bezstronnego testu końcowego [24, 25]. To też jeden okres rynkowy. Wyniki nie są prognozą live ani zielonym światłem do stakingu.

Benchmark pełnoskalowy jedzie na innej wersji danych i celu niż HPO. Pokazuje skalę, nie baseline jakości. Świadomie nie ekstrapolujemy CORR/MMC z 10% na pełny zbiór: więcej wierszy nie musi dawać liniowo lepszego wyniku.

Brakuje też kontrolowanego baseline'u i ablacji. Konfiguracje HPO różnią się wieloma elementami naraz, więc różnic nie przypiszemy samej rodzinie, selekcji cech ani neutralizacji. Porównanie rodzin jest opisowe. Nie mierzyliśmy też czasu człowieka ani AutoResearch vs TPE.

7.6 Rekomendacja wdrożeniowa i dalszy rozwój

Na pełny trening v5.3 bralibyśmy najpierw próbę 22: najlepszy CORR na późniejszym oknie i dodatnie MMC. Próbę 4 trzymamy jako wariant pod unikalność sygnału. Po treningu na pełnych danych obie receptury warto puścić przez kolejne rundy bez zmian konfiguracji - dopiero wtedy widać, co się trzyma live.

Dalej sens mają: wiele ziaren, kroczące okna przez więcej reżimów oraz ablacje routingu cech, selekcji wariancji i neutralizacji. AutoResearch jest gotowy jako warstwa automatyzacji, ale trzeba go mierzyć przy tym samym budżecie i przestrzeni co TPE. Po stronie produkcji naturalne jest też: harmonogram retreningu, walidacja artefaktu i monitoring dryfu per-era.

7.7 Wnioski końcowe

AlphaPulse przeprowadza eksperyment Numerai od danych i YAML-a, przez poprawną walidację czasową i HPO, po diagnostykę i eksport. W tej pracy system ogarnął trening pełnoskalowy i na v5.3 zostawił dwa niezdominowane profile: próba 22 (CORR Sharpe 0,4247, MMC Sharpe 0,2467 na późniejszym oknie) oraz próba 4 (MMC Sharpe 0,3063). To uzasadnia dalszą obserwację kandydatów, nieobciążoną prognozą live.

Najbardziej liczy się tu połączenie metodyki finansowej z inżynierią eksperymentów. Wynik to nie jedna liczba, tylko system, w którym da się odtworzyć decyzję, uczciwie porównać konfiguracje i dojść od hipotezy do `predict.pkl`. Na tym poziomie AlphaPulse spełnia cel konfigurowalnego AutoML pod Numerai.

Dodatek A

Materiały odtworzeniowe

A.1 Identyfikacja protokołu

Tabela A.1 zbiera identyfikatory potrzebne do powiązania wyników z kodem i danymi. Pełne rekordy prób pozostają w artefaktach repozytorium; załącznik nie zastępuje bazy SQLite, lecz wskazuje minimalny zestaw informacji potrzebny do audytu.

Tabela A.1: Identyfikatory końcowej serii eksperymentalnej

Element	Wartość
Wersja danych	Numerai v5.3
Cel	<code>target_ender_60</code>
Ziarno i udział danych	20260824; 10% zbioru uczącego
Podział czasowy	506 er uczenia; 16 er purge; 52 ery holdoutu; 88 er późniejszego przedziału
Wersja implementacji	commit <code>1a86893</code>
Identyfikator protokołu	<code>corrected-hpo-2026-08-v5</code>
Artefakty	<code>protocol.json</code> , <code>trials.db</code> , <code>optuna.db</code> , W&B

A.2 Receptury kandydatów

Tabela A.2 przedstawia parametry dwóch profili wybranych na froncie Pareto. Są to receptury do ponownego treningu i monitoringu, a nie modele o potwierdzonej jakości produkcyjnej.

Tabela A.2: Elementy konfiguracji kandydatów

Element	Próba 22	Próba 4
Rodzina modelu	LightGBM	CatBoost
Skalowanie	RobustScaler	StandardScaler
Cechy	989; medium+agility+midnight	970; sześć grup
Liczba iteracji	200 rund	zgodna z rekordem próby
Złożoność modelu	31 liści	głębokość 5
Tempo uczenia	0,0200	0,0314
Neutralizacja cech	0,2373	0,4947
Profil wyboru	najwyższy CORR na późniejszym przedziale	najwyższy MMC na późniejszym przedziale

A.3 Granice odtworzenia wyniku

Ponowne wykonanie poleceń z rozdziału 5 odtwarza procedurę i przestrzeń poszukiwania, ale nie gwarantuje identycznej kolejności prób na innym sprzęcie ani w innym zestawie wersji bibliotek. Do ścisłego audytu pojedynczego wyniku należy użyć zapisanej konfiguracji próby, skrótów danych i wersji kodu. Ponieważ kandydaci zostali wybrani na późniejszym przedziale, ich przyszłą jakość należy oceniać dopiero na kolejnych rundach, bez dalszego dostrajania konfiguracji.

Bibliografia

- [1] Palamabron, *AlphaPulse v0.6.0*. <https://github.com/Palamabron/AlphaPulse>.
- [2] Numerai, *Scoring - Numerai Tournament Documentation*. <https://docs.numer.ai/numera-tournament/scoring/>, [dostęp: 23.08.2026].
- [3] Numerai, *Correlation (CORR) - Numerai Tournament Documentation*. <https://docs.numer.ai/numera-tournament/scoring/correlation-corr>, [dostęp: 23.08.2026].
- [4] Numerai, *Meta Model Contribution (MMC) - Numerai Tournament Documentation*. <https://docs.numer.ai/numera-tournament/scoring/meta-model-contribution-mm-c>, [dostęp: 23.08.2026].
- [5] Numerai, *Staking - Numerai Tournament Documentation*. <https://docs.numer.ai/numera-tournament/staking>, [dostęp: 23.08.2026].
- [6] Numerai, *numera-tools 0.6.0*, Python Package Index, 10.06.2026. <https://pypi.org/project/numera-tools/0.6.0/>, [dostęp: 23.08.2026].
- [7] M. López de Prado, *Advances in Financial Machine Learning*. Wiley, 2018.
- [8] T. Akiba et al., “Optuna: A Next-generation Hyperparameter Optimization Framework”, *Proceedings of KDD*, 2019.
- [9] R. Liaw et al., “Tune: A Research Platform for Distributed Model Selection and Training”, *arXiv:1807.05118*, 2018.
- [10] X. He, K. Zhao and X. Chu, “AutoML: A Survey of the State-of-the-Art”, *Knowledge-Based Systems*, vol. 212, 2021.
- [11] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System”, *Proceedings of KDD*, 2016.
- [12] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”, *Advances in Neural Information Processing Systems*, 2017.
- [13] L. Prokhorenkova et al., “CatBoost: Unbiased Boosting with Categorical Features”, *Advances in Neural Information Processing Systems*, 2018.
- [14] D. H. Wolpert, “Stacked Generalization”, *Neural Networks*, vol. 5, no. 2, pp. 241-259, 1992.
- [15] N. Hollmann et al., “Accurate Predictions on Small Data with a Tabular Foundation Model”, *Nature*, vol. 637, pp. 319-326, 2025.
- [16] J. Qu, D. Holzmüller, G. Varoquaux and M. Le Morvan, “TabICL: A Tabular Foundation Model for In-Context Learning on Large Data”, *Proceedings of ICML*, 2025, arXiv:2502.05564.

- [17] L. Grinsztajn, E. Oyallon and G. Varoquaux, “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?”, *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [18] M. Feurer et al., “Efficient and Robust Automated Machine Learning”, *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [19] L. Kotthoff, C. Thornton, H. H. Hoos, F. Hutter and K. Leyton-Brown, “Auto-WEKA 2.0: Automatic Model Selection and Hyperparameter Optimization in WEKA”, *Journal of Machine Learning Research*, vol. 18, no. 25, pp. 1-5, 2017.
- [20] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization”, *Journal of Machine Learning Research*, vol. 13, pp. 281-305, 2012.
- [21] J. Pineau et al., “Improving Reproducibility in Machine Learning Research”, *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1-20, 2021.
- [22] C. Bergmeir, R. J. Hyndman and B. Koo, “A Note on the Validity of Cross-Validation for Evaluating Autoregressive Time Series Prediction”, *Computational Statistics and Data Analysis*, vol. 120, pp. 70-83, 2018.
- [23] S. Varma and R. Simon, “Bias in Error Estimation When Using Cross-Validation for Model Selection”, *BMC Bioinformatics*, vol. 7, article 91, 2006.
- [24] G. C. Cawley and N. L. C. Talbot, “On Over-Fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation”, *Journal of Machine Learning Research*, vol. 11, pp. 2079-2107, 2010.
- [25] D. H. Bailey, J. M. Borwein, M. López de Prado and Q. J. Zhu, “The Probability of Backtest Overfitting”, *Journal of Computational Finance*, vol. 20, no. 4, pp. 39-69, 2017.
- [26] M. Sugiyama, M. Krauledat and K.-R. Müller, “Covariate Shift Adaptation by Importance Weighted Cross Validation”, *Journal of Machine Learning Research*, vol. 8, pp. 985-1005, 2007.